DOCUMENT TECHNIQUE COMPLET

Système de Gestion des Autorisations de Produits Chimiques Réglementés

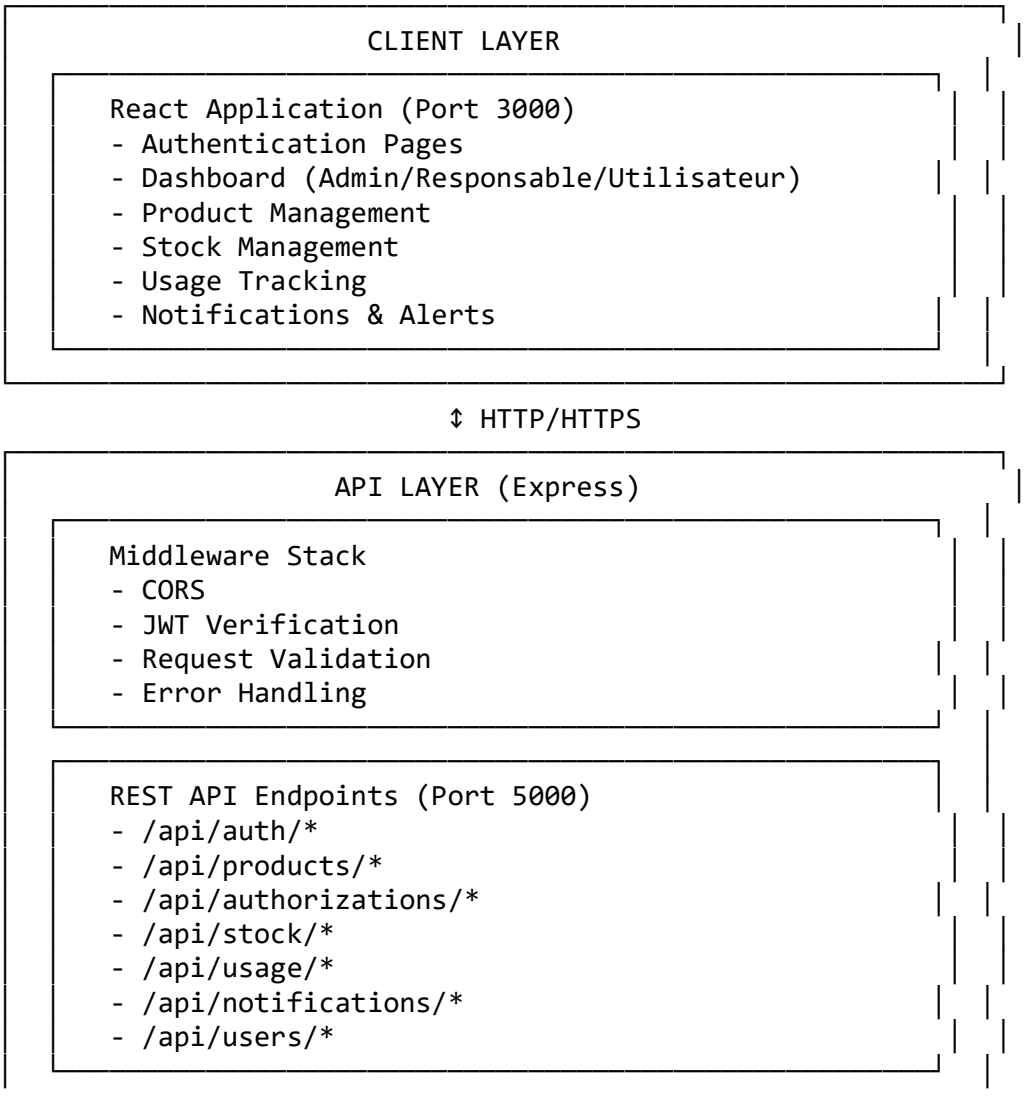
Stack Technologique

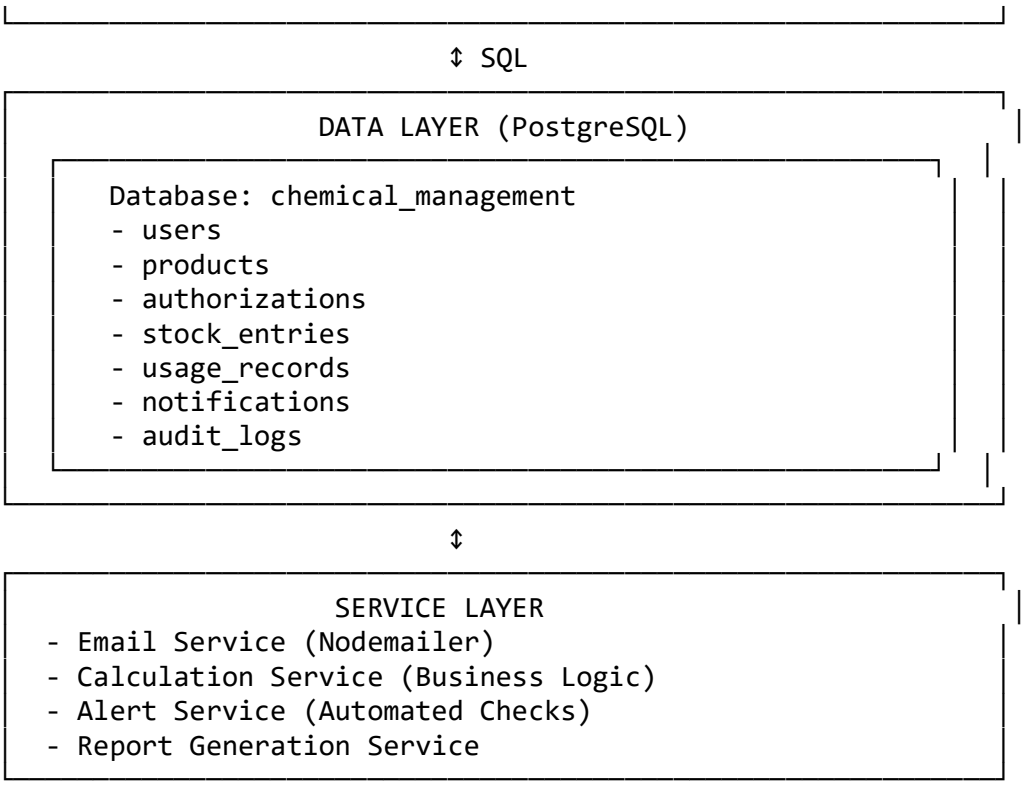
- **Backend**: Node.js + Express + PostgreSQL
- **Frontend**: React (JavaScript) + Tailwind CSS
- **Architecture**: REST API

---

1. ARCHITECTURE SYSTÈME

1.1 Architecture Globale





1.2 Flux de Données

Flux d'Authentification

1. User → POST /api/auth/login {email, password}
2. API → Verify credentials (bcrypt.compare)
3. API → Generate JWT token
4. API → Response {token, user, role}
5. Client → Store token in localStorage/sessionStorage
6. Client → Include token in Authorization header for subsequent requests

Flux de Création d'Autorisation

1. Responsable → POST /api/authorizations {productId, quantity, validityPeriod}
2. API → JWT Middleware (verify role)
3. API → Validate input data
4. API → Check product exists
5. API → Calculate expiration date
6. DB → INSERT INTO authorizations
7. Service → Schedule alert checks
8. API → Response {authorization}
9. Client → Update UI

Flux d'Utilisation de Produit

1. User → POST /api/usage {authorizationId, quantity, purpose}
2. API → JWT Middleware
3. API → Verify authorization exists and valid
4. API → Check remaining quantity
5. API → Calculate new totals

```
6. DB → BEGIN TRANSACTION
7. DB → INSERT INTO usage_records
8. DB → UPDATE authorizations SET used_quantity
9. Service → Check thresholds (20%, 50%)
10. Service → Create notifications if needed
11. DB → COMMIT
12. API → Response {usage, updatedAuthorization}
```

1.3 Sécurité

Authentification JWT

```
// Token Structure
{
  "userId": "uuid",
  "email": "user@example.com",
  "role": "ADMIN" | "RESPONSABLE" | "UTILISATEUR",
  "iat": 1234567890,
  "exp": 1234654290
}

// Token expiration: 24 hours
// Refresh token: 7 days
```

Hashage Mots de Passe

```
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 12;

// Création
const hashedPassword = await bcrypt.hash(plainPassword, SALT_ROUNDS);

// Vérification
const isValid = await bcrypt.compare(plainPassword, hashedPassword);
```

Rôles et Permissions

Rôle	Permissions
ADMIN	- Gestion utilisateurs- Gestion produits- Visualisation tous les rapports- Configuration système
RESPONSABLE	- Création autorisations- Gestion stock- Validation utilisations- Rapports de son département
UTILISATEUR	- Déclaration d'utilisation- Visualisation de ses autorisations- Consultation stock disponible

2. BASE DE DONNÉES POSTGRESQL

2.1 Schéma Complet

Table: users

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'RESPONSABLE', 'UTILISATEUR')),
    department VARCHAR(100),
    is_active BOOLEAN DEFAULT true,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by UUID REFERENCES users(id),
    CONSTRAINT email_format CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]
{2,}$')
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_department ON users(department);
```

Table: products

```
CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    cas_number VARCHAR(50) UNIQUE,
    chemical_formula VARCHAR(100),
    category VARCHAR(100) NOT NULL,
    regulatory_class VARCHAR(50) NOT NULL,
    unit VARCHAR(20) NOT NULL CHECK (unit IN ('kg', 'L', 'g', 'mL', 'unit')),
    minimum_stock NUMERIC(10, 3) DEFAULT 0,
    safety_instructions TEXT,
    storage_conditions TEXT,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by UUID REFERENCES users(id),
    CONSTRAINT positive_min_stock CHECK (minimum_stock >= 0)
);

CREATE INDEX idx_products_name ON products(name);
CREATE INDEX idx_products_cas ON products(cas_number);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_products_active ON products(is_active);
```

Table: authorizations

```
CREATE TABLE authorizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id),
    authorized_quantity NUMERIC(10, 3) NOT NULL,
    used_quantity NUMERIC(10, 3) DEFAULT 0,
    remaining_quantity NUMERIC(10, 3) GENERATED ALWAYS AS (authorized_quantity - used_qua
ntity) STORED,
    unit VARCHAR(20) NOT NULL,
    start_date DATE NOT NULL,
    validity_period_days INTEGER NOT NULL,
    expiration_date DATE NOT NULL,
    days_remaining INTEGER,
    percentage_used NUMERIC(5, 2) GENERATED ALWAYS AS (
        CASE
            WHEN authorized_quantity > 0 THEN (used_quantity / authorized_quantity * 100)
            ELSE 0
        END
    ) STORED,
    percentage_remaining NUMERIC(5, 2) GENERATED ALWAYS AS (
        CASE
            WHEN authorized_quantity > 0 THEN ((authorized_quantity - used_quantity) / au
thorized_quantity * 100)
            ELSE 0
        END
    ) STORED,
    status VARCHAR(20) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'EXPIRED', 'DEPLETED',
 'CANCELLED')),
    alert_level VARCHAR(20) DEFAULT 'GREEN' CHECK (alert_level IN ('GREEN', 'YELLOW', 'OR
ANGE', 'RED')),
    purpose TEXT,
    notes TEXT,
    approved_by UUID REFERENCES users(id),
    approved_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT positive_quantities CHECK (
        authorized_quantity > 0 AND
        used_quantity >= 0 AND
        used_quantity <= authorized_quantity
    ),
    CONSTRAINT valid_dates CHECK (expiration_date > start_date),
    CONSTRAINT valid_period CHECK (validity_period_days > 0)
);

CREATE INDEX idx_auth_product ON authorizations(product_id);
CREATE INDEX idx_auth_user ON authorizations(user_id);
CREATE INDEX idx_auth_status ON authorizations(status);
CREATE INDEX idx_auth_alert ON authorizations(alert_level);
```

```
CREATE INDEX idx_auth_expiration ON authorizations(expiration_date);
CREATE INDEX idx_auth_dates ON authorizations(start_date, expiration_date);
```

Table: stock_entries

```
CREATE TABLE stock_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    entry_type VARCHAR(20) NOT NULL CHECK (entry_type IN ('PURCHASE', 'RETURN', 'ADJUSTME
NT', 'TRANSFER')),
    quantity NUMERIC(10, 3) NOT NULL,
    unit VARCHAR(20) NOT NULL,
    batch_number VARCHAR(100),
    supplier VARCHAR(255),
    purchase_order VARCHAR(100),
    receipt_date DATE NOT NULL,
    manufacturing_date DATE,
    expiry_date DATE,
    unit_cost NUMERIC(10, 2),
    total_cost NUMERIC(12, 2) GENERATED ALWAYS AS (quantity * unit_cost) STORED,
    storage_location VARCHAR(100),
    notes TEXT,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT positive_quantity CHECK (quantity > 0),
    CONSTRAINT valid_expiry CHECK (expiry_date IS NULL OR expiry_date > manufacturing_dat
e)
);

CREATE INDEX idx_stock_product ON stock_entries(product_id);
CREATE INDEX idx_stock_type ON stock_entries(entry_type);
CREATE INDEX idx_stock_batch ON stock_entries(batch_number);
CREATE INDEX idx_stock_receipt ON stock_entries(receipt_date);
CREATE INDEX idx_stock_expiry ON stock_entries(expiry_date);
```

Table: usage_records

```
CREATE TABLE usage_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    authorization_id UUID NOT NULL REFERENCES authorizations(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id),
    product_id UUID NOT NULL REFERENCES products(id),
    quantity_used NUMERIC(10, 3) NOT NULL,
    unit VARCHAR(20) NOT NULL,
    usage_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    purpose TEXT NOT NULL,
    batch_number VARCHAR(100),
    project_reference VARCHAR(100),
    location VARCHAR(100),
    notes TEXT,
    validated_by UUID REFERENCES users(id),
    validated_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT positive_usage CHECK (quantity_used > 0)
);

CREATE INDEX idx_usage_auth ON usage_records(authorization_id);
CREATE INDEX idx_usage_user ON usage_records(user_id);
CREATE INDEX idx_usage_product ON usage_records(product_id);
CREATE INDEX idx_usage_date ON usage_records(usage_date);
CREATE INDEX idx_usage_validated ON usage_records(validated_by, validated_at);
```

Table: notifications

```
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    authorization_id UUID REFERENCES authorizations(id) ON DELETE CASCADE,
    type VARCHAR(50) NOT NULL CHECK (type IN (
        'AUTHORIZATION_EXPIRING',
        'AUTHORIZATION_EXPIRED',
        'QUANTITY_LOW',
        'QUANTITY_CRITICAL',
        'STOCK_LOW',
        'USAGE_SUBMITTED',
        'USAGE_VALIDATED',
        'SYSTEM_ALERT'
    )),
    priority VARCHAR(20) DEFAULT 'MEDIUM' CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH', 'C
RITICAL')),
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT false,
    read_at TIMESTAMP,
    email_sent BOOLEAN DEFAULT false,
    email_sent_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notif_user ON notifications(user_id);
CREATE INDEX idx_notif_unread ON notifications(user_id, is_read) WHERE is_read = false;
CREATE INDEX idx_notif_type ON notifications(type);
CREATE INDEX idx_notif_priority ON notifications(priority);
CREATE INDEX idx_notif_created ON notifications(created_at DESC);
```

Table: audit_logs

```
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID,
    old_values JSONB,
    new_values JSONB,
```

```
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_audit_user ON audit_logs(user_id);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_action ON audit_logs(action);
CREATE INDEX idx_audit_created ON audit_logs(created_at DESC);
```

2.2 Triggers

Trigger: Update days_remaining

```
CREATE OR REPLACE FUNCTION update_authorization_days_remaining()
RETURNS TRIGGER AS $$
BEGIN
    NEW.days_remaining := NEW.expiration_date - CURRENT_DATE;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_days_remaining
BEFORE INSERT OR UPDATE ON authorizations
FOR EACH ROW
EXECUTE FUNCTION update_authorization_days_remaining();
```

Trigger: Auto-update status and alert_level

```
CREATE OR REPLACE FUNCTION update_authorization_status_and_alerts()
RETURNS TRIGGER AS $$
BEGIN
    -- Update days_remaining
    NEW.days_remaining := NEW.expiration_date - CURRENT_DATE;

    -- Update status based on conditions
    IF NEW.days_remaining < 0 THEN
        NEW.status := 'EXPIRED';
    ELSIF NEW.remaining_quantity <= 0 THEN
        NEW.status := 'DEPLETED';
    ELSE
        NEW.status := 'ACTIVE';
    END IF;

    -- Update alert_level based on percentage_remaining
    IF NEW.percentage_remaining <= 20 OR NEW.days_remaining <= 7 THEN
        NEW.alert_level := 'RED';
    ELSIF NEW.percentage_remaining <= 50 OR NEW.days_remaining <= 30 THEN
        NEW.alert_level := 'ORANGE';
    ELSIF NEW.percentage_remaining <= 80 OR NEW.days_remaining <= 60 THEN
        NEW.alert_level := 'YELLOW';
    ELSE
        NEW.alert_level := 'GREEN';
```

```
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_status_alerts
BEFORE INSERT OR UPDATE ON authorizations
FOR EACH ROW
EXECUTE FUNCTION update_authorization_status_and_alerts();
```

Trigger: Create notifications on alert level change

```
CREATE OR REPLACE FUNCTION create_alert_notification()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if alert_level changed to ORANGE or RED
    IF (TG_OP = 'UPDATE' AND NEW.alert_level IN ('ORANGE', 'RED') AND
        (OLD.alert_level IS NULL OR OLD.alert_level != NEW.alert_level)) THEN

        -- Quantity alert
        IF NEW.percentage_remaining <= 50 THEN
            INSERT INTO notifications (user_id, authorization_id, type, priority, title,
message)
            VALUES (
                NEW.user_id,
                NEW.id,
                CASE
                    WHEN NEW.percentage_remaining <= 20 THEN 'QUANTITY_CRITICAL'
                    ELSE 'QUANTITY_LOW'
                END,
                CASE
                    WHEN NEW.percentage_remaining <= 20 THEN 'CRITICAL'
                    ELSE 'HIGH'
                END,
                'Alerte de quantité',
                format('Votre autorisation pour le produit a atteint %s%% de consommation.
 Il reste %s %s.',
                    ROUND(NEW.percentage_used, 1),
                    NEW.remaining_quantity,
                    NEW.unit
                )
            );
        END IF;

        -- Time alert
        IF NEW.days_remaining <= 30 THEN
            INSERT INTO notifications (user_id, authorization_id, type, priority, title,
message)
            VALUES (
                NEW.user_id,
```

```
                NEW.id,
                CASE
                    WHEN NEW.days_remaining <= 0 THEN 'AUTHORIZATION_EXPIRED'
                    WHEN NEW.days_remaining <= 7 THEN 'AUTHORIZATION_EXPIRING'
                    ELSE 'AUTHORIZATION_EXPIRING'
                END,
                CASE
                    WHEN NEW.days_remaining <= 7 THEN 'CRITICAL'
                    ELSE 'HIGH'
                END,
                'Alerte d''expiration',
                format('Votre autorisation expire dans %s jours (le %s).',
                    NEW.days_remaining,
                    TO_CHAR(NEW.expiration_date, 'DD/MM/YYYY')
                )
            );
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_create_alert_notification
AFTER INSERT OR UPDATE ON authorizations
FOR EACH ROW
EXECUTE FUNCTION create_alert_notification();
```

Trigger: Audit log for important changes

```
CREATE OR REPLACE FUNCTION log_authorization_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_logs (user_id, action, entity_type, entity_id, old_values, new_
values)
        VALUES (
            NEW.user_id,
            'UPDATE',
            'authorization',
            NEW.id,
            row_to_json(OLD),
            row_to_json(NEW)
        );
    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO audit_logs (user_id, action, entity_type, entity_id, new_values)
        VALUES (
            NEW.user_id,
            'CREATE',
            'authorization',
            NEW.id,
            row_to_json(NEW)
```

```
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_authorization_changes
AFTER INSERT OR UPDATE ON authorizations
FOR EACH ROW
EXECUTE FUNCTION log_authorization_changes();
```

Trigger: Update timestamp

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER trg_products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER trg_authorizations_updated_at
BEFORE UPDATE ON authorizations
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

2.3 Views Utiles

View: Active Authorizations Summary

```
CREATE OR REPLACE VIEW v_active_authorizations AS
SELECT
    a.id,
    a.user_id,
    u.first_name || ' ' || u.last_name AS user_name,
    u.email AS user_email,
    u.department,
    a.product_id,
    p.name AS product_name,
    p.cas_number,
    p.category AS product_category,
    a.authorized_quantity,
    a.used_quantity,
```

```
    a.remaining_quantity,
    a.unit,
    a.percentage_used,
    a.percentage_remaining,
    a.start_date,
    a.expiration_date,
    a.days_remaining,
    a.status,
    a.alert_level,
    a.purpose,
    a.created_at
FROM authorizations a
JOIN users u ON a.user_id = u.id
JOIN products p ON a.product_id = p.id
WHERE a.status = 'ACTIVE'
ORDER BY a.alert_level DESC, a.days_remaining ASC;
```

View: Stock Summary

```
CREATE OR REPLACE VIEW v_stock_summary AS
SELECT
    p.id AS product_id,
    p.name AS product_name,
    p.cas_number,
    p.category,
    p.unit,
    p.minimum_stock,
    COALESCE(SUM(se.quantity), 0) AS total_stock,
    COALESCE(SUM(CASE WHEN a.status = 'ACTIVE' THEN a.remaining_quantity ELSE 0 END), 0)
AS allocated_quantity,
    COALESCE(SUM(se.quantity), 0) - COALESCE(SUM(CASE WHEN a.status = 'ACTIVE' THEN a.rem
aining_quantity ELSE 0 END), 0) AS available_quantity,
    CASE
        WHEN COALESCE(SUM(se.quantity), 0) <= p.minimum_stock THEN 'LOW'
        WHEN COALESCE(SUM(se.quantity), 0) <= p.minimum_stock * 1.5 THEN 'MEDIUM'
        ELSE 'OK'
    END AS stock_status
FROM products p
LEFT JOIN stock_entries se ON p.id = se.product_id
LEFT JOIN authorizations a ON p.id = a.product_id
WHERE p.is_active = true
GROUP BY p.id, p.name, p.cas_number, p.category, p.unit, p.minimum_stock
ORDER BY stock_status DESC, p.name;
```

---

3. BACKEND NODE.JS

3.1 Structure des Dossiers

```
backend/
├── src/
```

```
├── config/
│   ├── database.js          # Configuration PostgreSQL
│   ├── jwt.js               # Configuration JWT
│   └── email.js             # Configuration Nodemailer
├── models/
│   ├── index.js             # Sequelize initialization
│   ├── User.js
│   ├── Product.js
│   ├── Authorization.js
│   ├── StockEntry.js
│   ├── UsageRecord.js
│   ├── Notification.js
│   └── AuditLog.js
├── middleware/
│   ├── auth.js              # JWT verification
│   ├── roleCheck.js         # Role-based access control
│   ├── validation.js        # Request validation
│   ├── errorHandler.js      # Global error handler
│   └── requestLogger.js     # Request logging
├── routes/
│   ├── index.js             # Route aggregator
│   ├── auth.routes.js
│   ├── user.routes.js
│   ├── product.routes.js
│   ├── authorization.routes.js
│   ├── stock.routes.js
│   ├── usage.routes.js
│   └── notification.routes.js
├── controllers/
│   ├── auth.controller.js
│   ├── user.controller.js
│   ├── product.controller.js
│   ├── authorization.controller.js
│   ├── stock.controller.js
│   ├── usage.controller.js
│   └── notification.controller.js
├── services/
│   ├── calculation.service.js    # Business logic calculations
│   ├── alert.service.js          # Alert generation
│   ├── email.service.js          # Email sending
│   ├── report.service.js         # Report generation
│   └── scheduler.service.js      # Cron jobs
├── validators/
│   ├── auth.validator.js
│   ├── user.validator.js
│   ├── product.validator.js
│   ├── authorization.validator.js
│   ├── stock.validator.js
│   └── usage.validator.js
├── utils/
│   ├── constants.js
```

```
│   │   ├── helpers.js
│   │   └── logger.js
│   └── app.js                    # Express app setup
├── tests/
│   ├── unit/
│   └── integration/
├── .env.example
├── .gitignore
├── package.json
└── server.js                     # Entry point
```

3.2 Configuration Files

package.json

```json
{
  "name": "chemical-management-backend",
  "version": "1.0.0",
  "description": "Backend API for Chemical Authorization Management System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest --coverage",
    "migrate": "sequelize-cli db:migrate",
    "migrate:undo": "sequelize-cli db:migrate:undo",
    "seed": "sequelize-cli db:seed:all"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0",
    "pg-hstore": "^2.3.4",
    "sequelize": "^6.32.1",
    "bcrypt": "^5.1.0",
    "jsonwebtoken": "^9.0.1",
    "dotenv": "^16.3.1",
    "cors": "^2.8.5",
    "helmet": "^7.0.0",
    "express-rate-limit": "^6.9.0",
    "express-validator": "^7.0.1",
    "nodemailer": "^6.9.4",
    "node-cron": "^3.0.2",
    "winston": "^3.10.0",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1",
    "jest": "^29.6.2",
    "supertest": "^6.3.3",
    "sequelize-cli": "^6.6.1"
  }
}
```

.env.example

```
# Server Configuration
NODE_ENV=development
PORT=5000
API_PREFIX=/api

# Database Configuration
DB_HOST=localhost
DB_PORT=5432
DB_NAME=chemical_management
DB_USER=postgres
DB_PASSWORD=your_password
DB_POOL_MIN=2
DB_POOL_MAX=10

# JWT Configuration
JWT_SECRET=your_super_secret_jwt_key_change_in_production
JWT_EXPIRES_IN=24h
JWT_REFRESH_SECRET=your_refresh_token_secret
JWT_REFRESH_EXPIRES_IN=7d

# Email Configuration (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your_email@gmail.com
SMTP_PASSWORD=your_app_password
SMTP_FROM_NAME=Chemical Management System
SMTP_FROM_EMAIL=noreply@company.com

# CORS Configuration
CORS_ORIGIN=http://localhost:3000

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100

# Alert Thresholds (from VBA logic)
ALERT_THRESHOLD_RED_PERCENTAGE=20
ALERT_THRESHOLD_ORANGE_PERCENTAGE=50
ALERT_THRESHOLD_YELLOW_PERCENTAGE=80
ALERT_THRESHOLD_RED_DAYS=7
ALERT_THRESHOLD_ORANGE_DAYS=30
ALERT_THRESHOLD_YELLOW_DAYS=60

# Scheduler
SCHEDULER_CHECK_ALERTS_CRON=0 */6 * * *
SCHEDULER_SEND_EMAIL_CRON=0 8 * * *
```

3.3 Database Configuration

config/database.js

```
const { Sequelize } = require('sequelize');
const winston = require('winston');

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: 'postgres',
    logging: (msg) => winston.info(msg),
    pool: {
      min: parseInt(process.env.DB_POOL_MIN) || 2,
      max: parseInt(process.env.DB_POOL_MAX) || 10,
      acquire: 30000,
      idle: 10000,
    },
    define: {
      timestamps: true,
      underscored: true,
      createdAt: 'created_at',
      updatedAt: 'updated_at',
    },
  }
);

const testConnection = async () => {
  try {
    await sequelize.authenticate();
    winston.info('Database connection established successfully.');
  } catch (error) {
    winston.error('Unable to connect to database:', error);
    process.exit(1);
  }
};

module.exports = { sequelize, testConnection };
```

3.4 Models (Sequelize)

models/User.js

```
const { DataTypes } = require('sequelize');
const bcrypt = require('bcrypt');
const { sequelize } = require('../config/database');

const User = sequelize.define('User', {
  id: {
```

```
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  email: {
    type: DataTypes.STRING(255),
    allowNull: false,
    unique: true,
    validate: {
      isEmail: true,
    },
  },
  passwordHash: {
    type: DataTypes.STRING(255),
    allowNull: false,
    field: 'password_hash',
  },
  firstName: {
    type: DataTypes.STRING(100),
    allowNull: false,
    field: 'first_name',
  },
  lastName: {
    type: DataTypes.STRING(100),
    allowNull: false,
    field: 'last_name',
  },
  role: {
    type: DataTypes.ENUM('ADMIN', 'RESPONSABLE', 'UTILISATEUR'),
    allowNull: false,
  },
  department: {
    type: DataTypes.STRING(100),
    allowNull: true,
  },
  isActive: {
    type: DataTypes.BOOLEAN,
    defaultValue: true,
    field: 'is_active',
  },
  lastLogin: {
    type: DataTypes.DATE,
    allowNull: true,
    field: 'last_login',
  },
}, {
  tableName: 'users',
  indexes: [
    { fields: ['email'] },
    { fields: ['role'] },
    { fields: ['department'] },
```

```
  ],
});

// Instance methods
User.prototype.validatePassword = async function(password) {
  return await bcrypt.compare(password, this.passwordHash);
};

User.prototype.toJSON = function() {
  const values = { ...this.get() };
  delete values.passwordHash;
  return values;
};

// Static methods
User.hashPassword = async (password) => {
  return await bcrypt.hash(password, 12);
};

// Hooks
User.beforeCreate(async (user) => {
  if (user.passwordHash) {
    user.passwordHash = await User.hashPassword(user.passwordHash);
  }
});

User.beforeUpdate(async (user) => {
  if (user.changed('passwordHash')) {
    user.passwordHash = await User.hashPassword(user.passwordHash);
  }
});

module.exports = User;
```

models/Product.js

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const Product = sequelize.define('Product', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  name: {
    type: DataTypes.STRING(255),
    allowNull: false,
  },
  casNumber: {
    type: DataTypes.STRING(50),
```

```
    unique: true,
    field: 'cas_number',
  },
  chemicalFormula: {
    type: DataTypes.STRING(100),
    field: 'chemical_formula',
  },
  category: {
    type: DataTypes.STRING(100),
    allowNull: false,
  },
  regulatoryClass: {
    type: DataTypes.STRING(50),
    allowNull: false,
    field: 'regulatory_class',
  },
  unit: {
    type: DataTypes.ENUM('kg', 'L', 'g', 'mL', 'unit'),
    allowNull: false,
  },
  minimumStock: {
    type: DataTypes.DECIMAL(10, 3),
    defaultValue: 0,
    field: 'minimum_stock',
    validate: {
      min: 0,
    },
  },
  safetyInstructions: {
    type: DataTypes.TEXT,
    field: 'safety_instructions',
  },
  storageConditions: {
    type: DataTypes.TEXT,
    field: 'storage_conditions',
  },
  isActive: {
    type: DataTypes.BOOLEAN,
    defaultValue: true,
    field: 'is_active',
  },
}, {
  tableName: 'products',
  indexes: [
    { fields: ['name'] },
    { fields: ['cas_number'] },
    { fields: ['category'] },
    { fields: ['is_active'] },
  ],
});
```

```
module.exports = Product;
```

models/Authorization.js

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const Authorization = sequelize.define('Authorization', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  productId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'product_id',
    references: {
      model: 'products',
      key: 'id',
    },
  },
  userId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'user_id',
    references: {
      model: 'users',
      key: 'id',
    },
  },
  authorizedQuantity: {
    type: DataTypes.DECIMAL(10, 3),
    allowNull: false,
    field: 'authorized_quantity',
    validate: {
      min: 0.001,
    },
  },
  usedQuantity: {
    type: DataTypes.DECIMAL(10, 3),
    defaultValue: 0,
    field: 'used_quantity',
    validate: {
      min: 0,
    },
  },
  remainingQuantity: {
    type: DataTypes.VIRTUAL,
    get() {
      return parseFloat(this.authorizedQuantity) - parseFloat(this.usedQuantity);
```

```
  },
},
unit: {
  type: DataTypes.STRING(20),
  allowNull: false,
},
startDate: {
  type: DataTypes.DATEONLY,
  allowNull: false,
  field: 'start_date',
},
validityPeriodDays: {
  type: DataTypes.INTEGER,
  allowNull: false,
  field: 'validity_period_days',
  validate: {
    min: 1,
  },
},
expirationDate: {
  type: DataTypes.DATEONLY,
  allowNull: false,
  field: 'expiration_date',
},
daysRemaining: {
  type: DataTypes.INTEGER,
  field: 'days_remaining',
},
percentageUsed: {
  type: DataTypes.VIRTUAL,
  get() {
    const authorized = parseFloat(this.authorizedQuantity);
    if (authorized === 0) return 0;
    return ((parseFloat(this.usedQuantity) / authorized) * 100).toFixed(2);
  },
},
percentageRemaining: {
  type: DataTypes.VIRTUAL,
  get() {
    return (100 - parseFloat(this.percentageUsed)).toFixed(2);
  },
},
status: {
  type: DataTypes.ENUM('ACTIVE', 'EXPIRED', 'DEPLETED', 'CANCELLED'),
  defaultValue: 'ACTIVE',
},
alertLevel: {
  type: DataTypes.ENUM('GREEN', 'YELLOW', 'ORANGE', 'RED'),
  defaultValue: 'GREEN',
  field: 'alert_level',
},
```

```
  purpose: {
    type: DataTypes.TEXT,
  },
  notes: {
    type: DataTypes.TEXT,
  },
  approvedBy: {
    type: DataTypes.UUID,
    field: 'approved_by',
    references: {
      model: 'users',
      key: 'id',
    },
  },
  approvedAt: {
    type: DataTypes.DATE,
    field: 'approved_at',
  },
}, {
  tableName: 'authorizations',
  indexes: [
    { fields: ['product_id'] },
    { fields: ['user_id'] },
    { fields: ['status'] },
    { fields: ['alert_level'] },
    { fields: ['expiration_date'] },
  ],
});

// Instance methods
Authorization.prototype.calculateExpirationDate = function() {
  const start = new Date(this.startDate);
  const expiration = new Date(start);
  expiration.setDate(start.getDate() + this.validityPeriodDays);
  return expiration.toISOString().split('T')[0];
};

Authorization.prototype.calculateDaysRemaining = function() {
  const today = new Date();
  const expiration = new Date(this.expirationDate);
  const diffTime = expiration - today;
  const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
  return diffDays;
};

Authorization.prototype.updateAlertLevel = function() {
  const percentRemaining = parseFloat(this.percentageRemaining);
  const daysLeft = this.calculateDaysRemaining();

  // RED: ≤20% OR ≤7 days
```

```
  if (percentRemaining <= 20 || daysLeft <= 7) {
    this.alertLevel = 'RED';
  }
  // ORANGE: ≤50% OR ≤30 days
  else if (percentRemaining <= 50 || daysLeft <= 30) {
    this.alertLevel = 'ORANGE';
  }
  // YELLOW: ≤80% OR ≤60 days
  else if (percentRemaining <= 80 || daysLeft <= 60) {
    this.alertLevel = 'YELLOW';
  }
  // GREEN: Everything else
  else {
    this.alertLevel = 'GREEN';
  }
};

Authorization.prototype.updateStatus = function() {
  const daysLeft = this.calculateDaysRemaining();

  if (daysLeft < 0) {
    this.status = 'EXPIRED';
  } else if (this.remainingQuantity <= 0) {
    this.status = 'DEPLETED';
  } else {
    this.status = 'ACTIVE';
  }
};

// Hooks
Authorization.beforeSave(async (authorization) => {
  // Calculate expiration date if not set
  if (!authorization.expirationDate && authorization.startDate && authorization.validityP
eriodDays) {
    authorization.expirationDate = authorization.calculateExpirationDate();
  }

  // Update days remaining
  authorization.daysRemaining = authorization.calculateDaysRemaining();

  // Update status
  authorization.updateStatus();

  // Update alert level
  authorization.updateAlertLevel();
});

module.exports = Authorization;
```

models/UsageRecord.js

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const UsageRecord = sequelize.define('UsageRecord', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  authorizationId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'authorization_id',
    references: {
      model: 'authorizations',
      key: 'id',
    },
  },
  userId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'user_id',
    references: {
      model: 'users',
      key: 'id',
    },
  },
  productId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'product_id',
    references: {
      model: 'products',
      key: 'id',
    },
  },
  quantityUsed: {
    type: DataTypes.DECIMAL(10, 3),
    allowNull: false,
    field: 'quantity_used',
    validate: {
      min: 0.001,
    },
  },
  unit: {
    type: DataTypes.STRING(20),
    allowNull: false,
  },
  usageDate: {
```

```
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW,
    field: 'usage_date',
  },
  purpose: {
    type: DataTypes.TEXT,
    allowNull: false,
  },
  batchNumber: {
    type: DataTypes.STRING(100),
    field: 'batch_number',
  },
  projectReference: {
    type: DataTypes.STRING(100),
    field: 'project_reference',
  },
  location: {
    type: DataTypes.STRING(100),
  },
  notes: {
    type: DataTypes.TEXT,
  },
  validatedBy: {
    type: DataTypes.UUID,
    field: 'validated_by',
    references: {
      model: 'users',
      key: 'id',
    },
  },
  validatedAt: {
    type: DataTypes.DATE,
    field: 'validated_at',
  },
}, {
  tableName: 'usage_records',
  indexes: [
    { fields: ['authorization_id'] },
    { fields: ['user_id'] },
    { fields: ['product_id'] },
    { fields: ['usage_date'] },
  ],
});

module.exports = UsageRecord;
```

3.5 Routes API Complètes

routes/auth.routes.js

```
const express = require('express');
const router = express.Router();
```

```
const authController = require('../controllers/auth.controller');
const { validateLogin, validateRegister } = require('../validators/auth.validator');

/**
 * @route   POST /api/auth/login
 * @desc    User login
 * @access  Public
 * @body    { email: string, password: string }
 * @return  { token: string, refreshToken: string, user: object }
 */
router.post('/login', validateLogin, authController.login);

/**
 * @route   POST /api/auth/register
 * @desc    Register new user (Admin only via middleware in main app)
 * @access  Private (Admin)
 * @body    { email, password, firstName, lastName, role, department }
 * @return  { user: object }
 */
router.post('/register', validateRegister, authController.register);

/**
 * @route   POST /api/auth/refresh
 * @desc    Refresh access token
 * @access  Public
 * @body    { refreshToken: string }
 * @return  { token: string }
 */
router.post('/refresh', authController.refreshToken);

/**
 * @route   POST /api/auth/logout
 * @desc    User logout
 * @access  Private
 * @header  Authorization: Bearer <token>
 * @return  { message: string }
 */
router.post('/logout', authController.logout);

/**
 * @route   POST /api/auth/forgot-password
 * @desc    Request password reset
 * @access  Public
 * @body    { email: string }
 * @return  { message: string }
 */
router.post('/forgot-password', authController.forgotPassword);

/**
 * @route   POST /api/auth/reset-password
```

```
 * @desc    Reset password with token
 * @access  Public
 * @body    { token: string, newPassword: string }
 * @return  { message: string }
 */
router.post('/reset-password', authController.resetPassword);

module.exports = router;
```

routes/authorization.routes.js

```
const express = require('express');
const router = express.Router();
const authorizationController = require('../controllers/authorization.controller');
const { authenticateToken } = require('../middleware/auth');
const { checkRole } = require('../middleware/roleCheck');
const { validateCreateAuthorization, validateUpdateUsage } = require('../validators/autho
rization.validator');

// All routes require authentication
router.use(authenticateToken);

/**
 * @route   GET /api/authorizations
 * @desc    Get all authorizations (filtered by role)
 * @access  Private
 * @query   ?status=ACTIVE&alertLevel=RED&userId=uuid&productId=uuid
 * @return  { authorizations: array, pagination: object }
 */
router.get('/', authorizationController.getAll);

/**
 * @route   GET /api/authorizations/my
 * @desc    Get current user's authorizations
 * @access  Private
 * @query   ?status=ACTIVE
 * @return  { authorizations: array }
 */
router.get('/my', authorizationController.getMyAuthorizations);

/**
 * @route   GET /api/authorizations/:id
 * @desc    Get authorization by ID
 * @access  Private
 * @params  id (UUID)
 * @return  { authorization: object, usageHistory: array }
 */
router.get('/:id', authorizationController.getById);

/**
 * @route   POST /api/authorizations
```

```
 * @desc    Create new authorization
 * @access  Private (Responsable, Admin)
 * @body    { productId, userId, authorizedQuantity, validityPeriodDays, purpose }
 * @return  { authorization: object }
 */
router.post(
  '/',
  checkRole(['RESPONSABLE', 'ADMIN']),
  validateCreateAuthorization,
  authorizationController.create
);

/**
 * @route   PUT /api/authorizations/:id
 * @desc    Update authorization
 * @access  Private (Responsable, Admin)
 * @params  id (UUID)
 * @body    { authorizedQuantity, validityPeriodDays, notes }
 * @return  { authorization: object }
 */
router.put(
  '/:id',
  checkRole(['RESPONSABLE', 'ADMIN']),
  authorizationController.update
);

/**
 * @route   DELETE /api/authorizations/:id
 * @desc    Cancel authorization
 * @access  Private (Responsable, Admin)
 * @params  id (UUID)
 * @return  { message: string }
 */
router.delete(
  '/:id',
  checkRole(['RESPONSABLE', 'ADMIN']),
  authorizationController.cancel
);

/**
 * @route   GET /api/authorizations/stats/dashboard
 * @desc    Get authorization statistics
 * @access  Private
 * @return  { total, active, expired, byAlertLevel, byStatus }
 */
router.get('/stats/dashboard', authorizationController.getStatistics);

/**
 * @route   GET /api/authorizations/:id/check-overage
 * @desc    Check if usage would exceed authorization
```

```
 * @access  Private
 * @params  id (UUID)
 * @query   ?quantity=number
 * @return  { allowed: boolean, remaining: number, message: string }
 */
router.get('/:id/check-overage', authorizationController.checkOverage);

module.exports = router;
```

routes/usage.routes.js

```
const express = require('express');
const router = express.Router();
const usageController = require('../controllers/usage.controller');
const { authenticateToken } = require('../middleware/auth');
const { checkRole } = require('../middleware/roleCheck');
const { validateCreateUsage } = require('../validators/usage.validator');

router.use(authenticateToken);

/**
 * @route   GET /api/usage
 * @desc    Get all usage records
 * @access  Private (Admin, Responsable)
 * @query   ?userId=uuid&productId=uuid&startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
 * @return  { usageRecords: array, pagination: object }
 */
router.get(
  '/',
  checkRole(['ADMIN', 'RESPONSABLE']),
  usageController.getAll
);

/**
 * @route   GET /api/usage/my
 * @desc    Get current user's usage history
 * @access  Private
 * @return  { usageRecords: array }
 */
router.get('/my', usageController.getMyUsage);

/**
 * @route   POST /api/usage
 * @desc    Record new usage
 * @access  Private
 * @body    { authorizationId, quantityUsed, purpose, batchNumber, location }
 * @return  { usageRecord: object, updatedAuthorization: object }
 */
router.post(
  '/',
  validateCreateUsage,
```

```
  usageController.create
);

/**
 * @route   PUT /api/usage/:id/validate
 * @desc    Validate usage record
 * @access  Private (Responsable, Admin)
 * @params  id (UUID)
 * @return  { usageRecord: object }
 */
router.put(
  '/:id/validate',
  checkRole(['RESPONSABLE', 'ADMIN']),
  usageController.validate
);

/**
 * @route   GET /api/usage/report
 * @desc    Generate usage report
 * @access  Private (Responsable, Admin)
 * @query   ?startDate&endDate&userId&productId&format=json|csv
 * @return  Report data or CSV file
 */
router.get(
  '/report',
  checkRole(['RESPONSABLE', 'ADMIN']),
  usageController.generateReport
);

module.exports = router;
```

3.6 Controllers

controllers/authorization.controller.js

```
const { Authorization, Product, User, UsageRecord } = require('../models');
const { Op } = require('sequelize');
const calculationService = require('../services/calculation.service');
const alertService = require('../services/alert.service');

class AuthorizationController {
  /**
   * Get all authorizations with filters
   */
  async getAll(req, res, next) {
    try {
      const {
        status,
        alertLevel,
        userId,
        productId,
        page = 1,
```

```
    limit = 20,
    sortBy = 'createdAt',
    sortOrder = 'DESC'
  } = req.query;

  const where = {};

  // Role-based filtering
  if (req.user.role === 'UTILISATEUR') {
    where.userId = req.user.id;
  } else if (req.user.role === 'RESPONSABLE') {
    // Responsable sees their department
    const departmentUsers = await User.findAll({
      where: { department: req.user.department },
      attributes: ['id']
    });
    where.userId = { [Op.in]: departmentUsers.map(u => u.id) };
  }

  // Apply filters
  if (status) where.status = status;
  if (alertLevel) where.alertLevel = alertLevel;
  if (userId) where.userId = userId;
  if (productId) where.productId = productId;

  const offset = (page - 1) * limit;

  const { count, rows } = await Authorization.findAndCountAll({
    where,
    include: [
      {
        model: Product,
        attributes: ['id', 'name', 'casNumber', 'unit']
      },
      {
        model: User,
        as: 'user',
        attributes: ['id', 'firstName', 'lastName', 'email', 'department']
      }
    ],
    limit: parseInt(limit),
    offset: offset,
    order: [[sortBy, sortOrder]]
  });

  res.json({
    success: true,
    data: rows,
    pagination: {
      total: count,
```

```
        page: parseInt(page),
        pages: Math.ceil(count / limit),
        limit: parseInt(limit)
      }
    });
  } catch (error) {
    next(error);
  }
}

/**
 * Create new authorization
 */
async create(req, res, next) {
  try {
    const {
      productId,
      userId,
      authorizedQuantity,
      validityPeriodDays,
      purpose,
      notes
    } = req.body;

    // Verify product exists
    const product = await Product.findByPk(productId);
    if (!product) {
      return res.status(404).json({
        success: false,
        message: 'Produit non trouvé'
      });
    }

    // Verify user exists
    const user = await User.findByPk(userId);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'Utilisateur non trouvé'
      });
    }

    // Calculate dates (matching VBA logic)
    const startDate = new Date().toISOString().split('T')[0];
    const expirationDate = calculationService.calculateExpirationDate(
      startDate,
      validityPeriodDays
    );

    // Create authorization
```

```
      const authorization = await Authorization.create({
        productId,
        userId,
        authorizedQuantity,
        unit: product.unit,
        startDate,
        validityPeriodDays,
        expirationDate,
        purpose,
        notes,
        approvedBy: req.user.id,
        approvedAt: new Date(),
        status: 'ACTIVE',
        alertLevel: 'GREEN',
        usedQuantity: 0
      });

      // Load full authorization with associations
      const fullAuthorization = await Authorization.findByPk(authorization.id, {
        include: [
          { model: Product },
          { model: User, as: 'user' }
        ]
      });

      // Create notification for user
      await alertService.createNotification({
        userId: userId,
        authorizationId: authorization.id,
        type: 'AUTHORIZATION_CREATED',
        priority: 'MEDIUM',
        title: 'Nouvelle autorisation',
        message: `Vous avez reçu une autorisation pour ${product.name} (${authorizedQuant
ity} ${product.unit})`
      });

      res.status(201).json({
        success: true,
        data: fullAuthorization,
        message: 'Autorisation créée avec succès'
      });
    } catch (error) {
      next(error);
    }
  }

  /**
   * Check if quantity would exceed authorization (VBA: ControleDepassement)
   */
  async checkOverage(req, res, next) {
```

```
    try {
      const { id } = req.params;
      const { quantity } = req.query;

      const authorization = await Authorization.findByPk(id);
      if (!authorization) {
        return res.status(404).json({
          success: false,
          message: 'Autorisation non trouvée'
        });
      }

      const requestedQuantity = parseFloat(quantity);
      const remaining = authorization.remainingQuantity;

      const result = {
        allowed: requestedQuantity <= remaining,
        remaining: remaining,
        requested: requestedQuantity,
        wouldExceedBy: Math.max(0, requestedQuantity - remaining)
      };

      if (!result.allowed) {
        result.message = `Quantité demandée (${requestedQuantity}) dépasse la quantité au
torisée restante (${remaining})`;
      } else {
        result.message = 'Utilisation autorisée';
      }

      res.json({
        success: true,
        data: result
      });
    } catch (error) {
      next(error);
    }
  }

  /**
   * Get authorization statistics
   */
  async getStatistics(req, res, next) {
    try {
      const where = {};

      // Apply role-based filtering
      if (req.user.role === 'UTILISATEUR') {
        where.userId = req.user.id;
      }
```

```
      const [total, active, expired, depleted] = await Promise.all([
        Authorization.count({ where }),
        Authorization.count({ where: { ...where, status: 'ACTIVE' } }),
        Authorization.count({ where: { ...where, status: 'EXPIRED' } }),
        Authorization.count({ where: { ...where, status: 'DEPLETED' } })
      ]);

      const byAlertLevel = await Authorization.findAll({
        where: { ...where, status: 'ACTIVE' },
        attributes: [
          'alertLevel',
          [sequelize.fn('COUNT', sequelize.col('id')), 'count']
        ],
        group: ['alertLevel']
      });

      const alertLevelStats = {
        GREEN: 0,
        YELLOW: 0,
        ORANGE: 0,
        RED: 0
      };

      byAlertLevel.forEach(item => {
        alertLevelStats[item.alertLevel] = parseInt(item.get('count'));
      });

      res.json({
        success: true,
        data: {
          total,
          byStatus: {
            active,
            expired,
            depleted
          },
          byAlertLevel: alertLevelStats
        }
      });
    } catch (error) {
      next(error);
    }
  }
}

module.exports = new AuthorizationController();
```

controllers/usage.controller.js

```
const { UsageRecord, Authorization, Product, User } = require('../models');
const { sequelize } = require('../config/database');
```

```
const alertService = require('../services/alert.service');
const calculationService = require('../services/calculation.service');

class UsageController {
  /**
   * Create new usage record (matching VBA: EnregistrerUtilisation)
   */
  async create(req, res, next) {
    const transaction = await sequelize.transaction();

    try {
      const {
        authorizationId,
        quantityUsed,
        purpose,
        batchNumber,
        projectReference,
        location,
        notes
      } = req.body;

      // Get authorization with lock
      const authorization = await Authorization.findByPk(authorizationId, {
        include: [{ model: Product }],
        lock: transaction.LOCK.UPDATE,
        transaction
      });

      if (!authorization) {
        await transaction.rollback();
        return res.status(404).json({
          success: false,
          message: 'Autorisation non trouvée'
        });
      }

      // Check authorization belongs to user (unless admin/responsable)
      if (req.user.role === 'UTILISATEUR' && authorization.userId !== req.user.id) {
        await transaction.rollback();
        return res.status(403).json({
          success: false,
          message: 'Cette autorisation ne vous appartient pas'
        });
      }

      // Check authorization is active
      if (authorization.status !== 'ACTIVE') {
        await transaction.rollback();
        return res.status(400).json({
          success: false,
```

```
          message: `Autorisation ${authorization.status.toLowerCase()}. Utilisation impos
sible.`
        });
      }

      // Check if quantity exceeds remaining (VBA: ControleDepassement)
      const remaining = parseFloat(authorization.remainingQuantity);
      const requested = parseFloat(quantityUsed);

      if (requested > remaining) {
        await transaction.rollback();
        return res.status(400).json({
          success: false,
          message: `Quantité demandée (${requested}) dépasse la quantité restante (${rema
ining})`,
          data: {
            remaining,
            requested,
            exceeded: requested - remaining
          }
        });
      }

      // Create usage record
      const usageRecord = await UsageRecord.create({
        authorizationId,
        userId: req.user.id,
        productId: authorization.productId,
        quantityUsed: requested,
        unit: authorization.unit,
        purpose,
        batchNumber,
        projectReference,
        location,
        notes,
        usageDate: new Date()
      }, { transaction });

      // Update authorization used_quantity
      const newUsedQuantity = parseFloat(authorization.usedQuantity) + requested;
      await authorization.update(
        { usedQuantity: newUsedQuantity },
        { transaction }
      );

      // Reload to get updated calculated fields
      await authorization.reload({ transaction });

      // Check thresholds and create alerts (VBA: VerificationSeuils)
      await this.checkThresholdsAndAlert(authorization, transaction);
```

```
      await transaction.commit();

      // Load complete usage record
      const completeUsage = await UsageRecord.findByPk(usageRecord.id, {
        include: [
          { model: Product },
          { model: User, as: 'user', attributes: ['id', 'firstName', 'lastName'] }
        ]
      });

      res.status(201).json({
        success: true,
        data: {
          usageRecord: completeUsage,
          updatedAuthorization: {
            id: authorization.id,
            usedQuantity: authorization.usedQuantity,
            remainingQuantity: authorization.remainingQuantity,
            percentageUsed: authorization.percentageUsed,
            percentageRemaining: authorization.percentageRemaining,
            alertLevel: authorization.alertLevel,
            status: authorization.status
          }
        },
        message: 'Utilisation enregistrée avec succès'
      });
    } catch (error) {
      await transaction.rollback();
      next(error);
    }
  }

  /**
   * Check thresholds and create alerts (VBA logic)
   */
  async checkThresholdsAndAlert(authorization, transaction = null) {
    const percentRemaining = parseFloat(authorization.percentageRemaining);
    const daysRemaining = authorization.daysRemaining;

    const alerts = [];

    // Quantity alerts
    if (percentRemaining <= 20) {
      alerts.push({
        type: 'QUANTITY_CRITICAL',
        priority: 'CRITICAL',
        title: 'Alerte critique - Quantité restante',
        message: `Votre autorisation pour ${authorization.Product.name} a atteint ${autho
rization.percentageUsed}% de consommation. Il ne reste que ${authorization.remainingQuant
```

```
ity} ${authorization.unit}.`
      });
    } else if (percentRemaining <= 50) {
      alerts.push({
        type: 'QUANTITY_LOW',
        priority: 'HIGH',
        title: 'Alerte - Quantité faible',
        message: `Votre autorisation pour ${authorization.Product.name} a atteint ${autho
rization.percentageUsed}% de consommation. Il reste ${authorization.remainingQuantity}
${authorization.unit}.`
      });
    }

    // Time alerts
    if (daysRemaining <= 7 && daysRemaining >= 0) {
      alerts.push({
        type: 'AUTHORIZATION_EXPIRING',
        priority: 'CRITICAL',
        title: 'Autorisation expire bientôt',
        message: `Votre autorisation pour ${authorization.Product.name} expire dans ${day
sRemaining} jour(s).`
      });
    } else if (daysRemaining <= 30 && daysRemaining > 7) {
      alerts.push({
        type: 'AUTHORIZATION_EXPIRING',
        priority: 'HIGH',
        title: 'Autorisation expire dans 30 jours',
        message: `Votre autorisation pour ${authorization.Product.name} expire le ${new D
ate(authorization.expirationDate).toLocaleDateString('fr-FR')}.`
      });
    }

    // Create notifications
    for (const alert of alerts) {
      await alertService.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        ...alert
      }, transaction);
    }
  }

  /**
   * Get user's usage history
   */
  async getMyUsage(req, res, next) {
    try {
      const { startDate, endDate, productId } = req.query;

      const where = { userId: req.user.id };
```

```
      if (startDate || endDate) {
        where.usageDate = {};
        if (startDate) where.usageDate[Op.gte] = new Date(startDate);
        if (endDate) where.usageDate[Op.lte] = new Date(endDate);
      }

      if (productId) where.productId = productId;

      const usageRecords = await UsageRecord.findAll({
        where,
        include: [
          {
            model: Product,
            attributes: ['id', 'name', 'casNumber', 'unit']
          },
          {
            model: Authorization,
            attributes: ['id', 'authorizedQuantity', 'status']
          }
        ],
        order: [['usageDate', 'DESC']]
      });

      res.json({
        success: true,
        data: usageRecords
      });
    } catch (error) {
      next(error);
    }
  }
}

module.exports = new UsageController();
```

3.7 Services

services/calculation.service.js

```
/**
 * Calculation Service
 * Implements VBA business logic for dates, percentages, and thresholds
 */

class CalculationService {
  /**
   * Calculate expiration date (VBA: CalculDateEcheance)
   * @param {string|Date} startDate - Start date
   * @param {number} validityPeriodDays - Validity period in days
   * @returns {string} Expiration date in YYYY-MM-DD format
```

```
 */
calculateExpirationDate(startDate, validityPeriodDays) {
  const start = new Date(startDate);
  const expiration = new Date(start);
  expiration.setDate(start.getDate() + parseInt(validityPeriodDays));
  return expiration.toISOString().split('T')[0];
}

/**
 * Calculate days remaining until expiration (VBA: CalculJoursRestants)
 * @param {string|Date} expirationDate - Expiration date
 * @returns {number} Days remaining (negative if expired)
 */
calculateDaysRemaining(expirationDate) {
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const expiration = new Date(expirationDate);
  expiration.setHours(0, 0, 0, 0);

  const diffTime = expiration - today;
  const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
  return diffDays;
}

/**
 * Calculate percentage used (VBA: CalculPourcentageAcquis)
 * @param {number} usedQuantity - Quantity used
 * @param {number} authorizedQuantity - Total authorized quantity
 * @returns {number} Percentage used (0-100)
 */
calculatePercentageUsed(usedQuantity, authorizedQuantity) {
  if (authorizedQuantity === 0) return 0;
  const percentage = (usedQuantity / authorizedQuantity) * 100;
  return Math.round(percentage * 100) / 100; // Round to 2 decimals
}

/**
 * Calculate percentage remaining (VBA: CalculPourcentageRestant)
 * @param {number} usedQuantity - Quantity used
 * @param {number} authorizedQuantity - Total authorized quantity
 * @returns {number} Percentage remaining (0-100)
 */
calculatePercentageRemaining(usedQuantity, authorizedQuantity) {
  const percentageUsed = this.calculatePercentageUsed(usedQuantity, authorizedQuantity);
  return 100 - percentageUsed;
}

/**
 * Determine alert level based on thresholds (VBA: DeterminerNiveauAlerte)
```

```
   * @param {number} percentageRemaining - Percentage remaining
   * @param {number} daysRemaining - Days remaining
   * @returns {string} Alert level: 'GREEN', 'YELLOW', 'ORANGE', 'RED'
   */
  determineAlertLevel(percentageRemaining, daysRemaining) {
    // RED: <= 20% OR <= 7 days
    if (percentageRemaining <= 20 || daysRemaining <= 7) {
      return 'RED';
    }
    // ORANGE: <= 50% OR <= 30 days
    else if (percentageRemaining <= 50 || daysRemaining <= 30) {
      return 'ORANGE';
    }
    // YELLOW: <= 80% OR <= 60 days
    else if (percentageRemaining <= 80 || daysRemaining <= 60) {
      return 'YELLOW';
    }
    // GREEN: Everything else
    else {
      return 'GREEN';
    }
  }

  /**
   * Determine status based on conditions (VBA: DeterminerStatut)
   * @param {number} daysRemaining - Days remaining
   * @param {number} remainingQuantity - Quantity remaining
   * @returns {string} Status: 'ACTIVE', 'EXPIRED', 'DEPLETED'
   */
  determineStatus(daysRemaining, remainingQuantity) {
    if (daysRemaining < 0) {
      return 'EXPIRED';
    } else if (remainingQuantity <= 0) {
      return 'DEPLETED';
    } else {
      return 'ACTIVE';
    }
  }

  /**
   * Check if quantity exceeds authorization (VBA: ControleDepassement)
   * @param {number} requestedQuantity - Requested quantity
   * @param {number} remainingQuantity - Available quantity
   * @returns {object} { allowed: boolean, exceeded: number }
   */
  checkOverage(requestedQuantity, remainingQuantity) {
    const allowed = requestedQuantity <= remainingQuantity;
    const exceeded = Math.max(0, requestedQuantity - remainingQuantity);

    return {
```

```
      allowed,
      exceeded,
      remaining: remainingQuantity,
      requested: requestedQuantity
    };
  }

  /**
   * Format date for display (French format)
   * @param {string|Date} date - Date to format
   * @returns {string} Formatted date (DD/MM/YYYY)
   */
  formatDate(date) {
    const d = new Date(date);
    const day = String(d.getDate()).padStart(2, '0');
    const month = String(d.getMonth() + 1).padStart(2, '0');
    const year = d.getFullYear();
    return `${day}/${month}/${year}`;
  }

  /**
   * Get alert color based on level
   * @param {string} alertLevel - Alert level
   * @returns {string} Color code
   */
  getAlertColor(alertLevel) {
    const colors = {
      'GREEN': '#10B981',   // Tailwind green-500
      'YELLOW': '#F59E0B',  // Tailwind yellow-500
      'ORANGE': '#F97316',  // Tailwind orange-500
      'RED': '#EF4444'      // Tailwind red-500
    };
    return colors[alertLevel] || colors.GREEN;
  }
}

module.exports = new CalculationService();
```

services/alert.service.js

```
const { Notification, Authorization, Product, User } = require('../models');
const emailService = require('./email.service');
const calculationService = require('./calculation.service');

class AlertService {
  /**
   * Create notification
   */
  async createNotification(data, transaction = null) {
    const notification = await Notification.create(data, { transaction });
```

```
  // Send email if priority is HIGH or CRITICAL
  if (['HIGH', 'CRITICAL'].includes(data.priority)) {
    // Don't wait for email to complete
    this.sendEmailNotification(notification).catch(err => {
      console.error('Email notification failed:', err);
    });
  }

  return notification;
}

/**
 * Send email notification
 */
async sendEmailNotification(notification) {
  const user = await User.findByPk(notification.userId);
  if (!user || !user.email) return;

  await emailService.sendEmail({
    to: user.email,
    subject: `[${notification.priority}] ${notification.title}`,
    html: this.generateEmailTemplate(notification, user)
  });

  await notification.update({
    emailSent: true,
    emailSentAt: new Date()
  });
}

/**
 * Check all active authorizations for alerts
 */
async checkAllAuthorizationsForAlerts() {
  const activeAuthorizations = await Authorization.findAll({
    where: { status: 'ACTIVE' },
    include: [{ model: Product }, { model: User }]
  });

  for (const auth of activeAuthorizations) {
    await this.checkAuthorizationAlerts(auth);
  }
}

/**
 * Check single authorization for alerts
 */
async checkAuthorizationAlerts(authorization) {
  const percentRemaining = parseFloat(authorization.percentageRemaining);
  const daysRemaining = calculationService.calculateDaysRemaining(authorization.expirat
```

```
ionDate);

    // Check if we already sent alert for this level
    const existingNotification = await Notification.findOne({
      where: {
        authorizationId: authorization.id,
        createdAt: {
          [Op.gte]: new Date(Date.now() - 24 * 60 * 60 * 1000) // Last 24 hours
        }
      },
      order: [['createdAt', 'DESC']]
    });

    // Quantity critical (≤20%)
    if (percentRemaining <= 20 && (!existingNotification || existingNotification.type !==
 'QUANTITY_CRITICAL')) {
      await this.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        type: 'QUANTITY_CRITICAL',
        priority: 'CRITICAL',
        title: 'Alerte critique - Quantité restante',
        message: `Votre autorisation pour ${authorization.Product.name} a atteint ${autho
rization.percentageUsed}% de consommation. Il ne reste que ${authorization.remainingQuant
ity} ${authorization.unit}.`
      });
    }
    // Quantity low (≤50%)
    else if (percentRemaining <= 50 && percentRemaining > 20 && (!existingNotification ||
 !['QUANTITY_CRITICAL', 'QUANTITY_LOW'].includes(existingNotification.type))) {
      await this.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        type: 'QUANTITY_LOW',
        priority: 'HIGH',
        title: 'Alerte - Quantité faible',
        message: `Votre autorisation pour ${authorization.Product.name} a atteint ${autho
rization.percentageUsed}% de consommation. Il reste ${authorization.remainingQuantity}
${authorization.unit}.`
      });
    }

    // Expiration alerts
    if (daysRemaining <= 7 && daysRemaining >= 0) {
      await this.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        type: 'AUTHORIZATION_EXPIRING',
        priority: 'CRITICAL',
        title: 'Autorisation expire dans 7 jours',
```

```
        message: `Votre autorisation pour ${authorization.Product.name} expire dans ${day
sRemaining} jour(s) le ${calculationService.formatDate(authorization.expirationDate)}.`
      });
    } else if (daysRemaining <= 30 && daysRemaining > 7) {
      await this.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        type: 'AUTHORIZATION_EXPIRING',
        priority: 'HIGH',
        title: 'Autorisation expire dans 30 jours',
        message: `Votre autorisation pour ${authorization.Product.name} expire le ${calcu
lationService.formatDate(authorization.expirationDate)}.`
      });
    } else if (daysRemaining < 0) {
      await this.createNotification({
        userId: authorization.userId,
        authorizationId: authorization.id,
        type: 'AUTHORIZATION_EXPIRED',
        priority: 'CRITICAL',
        title: 'Autorisation expirée',
        message: `Votre autorisation pour ${authorization.Product.name} a expiré le ${cal
culationService.formatDate(authorization.expirationDate)}.`
      });
    }
  }

  /**
   * Generate email template
   */
  generateEmailTemplate(notification, user) {
    const priorityColors = {
      'LOW': '#10B981',
      'MEDIUM': '#F59E0B',
      'HIGH': '#F97316',
      'CRITICAL': '#EF4444'
    };

    return `
      <!DOCTYPE html>
      <html>
      <head>
        <meta charset="UTF-8">
        <style>
          body { font-family: Arial, sans-serif; line-height: 1.6; color: #333; }
          .container { max-width: 600px; margin: 0 auto; padding: 20px; }
          .header { background: ${priorityColors[notification.priority]}; color: white; p
adding: 20px; border-radius: 5px 5px 0 0; }
          .content { background: #f9fafb; padding: 20px; border: 1px solid #e5e7eb; borde
r-radius: 0 0 5px 5px; }
          .button { display: inline-block; padding: 10px 20px; background: #001965; color:
 white; text-decoration: none; border-radius: 5px; margin-top: 15px; }
```

```
          .footer { margin-top: 20px; font-size: 12px; color: #6b7280; }
        </style>
      </head>
      <body>
        <div class="container">
          <div class="header">
            <h2>${notification.title}</h2>
          </div>
          <div class="content">
            <p>Bonjour ${user.firstName} ${user.lastName},</p>
            <p>${notification.message}</p>
            <a href="${process.env.FRONTEND_URL}/authorizations" class="button">Voir mes
autorisations</a>
          </div>
          <div class="footer">
            <p>Ceci est un message automatique, merci de ne pas y répondre.</p>
            <p>© ${new Date().getFullYear()} Chemical Management System</p>
          </div>
        </div>
      </body>
      </html>
    `;
  }
}

module.exports = new AlertService();
```

---

4. FRONTEND REACT

4.1 Structure des Dossiers

```
frontend/
├── public/
│   ├── index.html
│   └── favicon.ico
├── src/
│   ├── api/
│   │   ├── axios.js              # Axios configuration
│   │   ├── auth.api.js
│   │   ├── authorization.api.js
│   │   ├── product.api.js
│   │   ├── usage.api.js
│   │   └── notification.api.js
│   ├── components/
│   │   ├── common/
│   │   │   ├── Button.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Select.jsx
│   │   │   ├── Modal.jsx
```

```
│   │   │   ├── Table.jsx
│   │   │   ├── Card.jsx
│   │   │   ├── Badge.jsx
│   │   │   ├── Alert.jsx
│   │   │   ├── LoadingSpinner.jsx
│   │   │   └── Pagination.jsx
│   │   ├── layout/
│   │   │   ├── Header.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── Footer.jsx
│   │   │   └── Layout.jsx
│   │   ├── auth/
│   │   │   ├── LoginForm.jsx
│   │   │   ├── ProtectedRoute.jsx
│   │   │   └── RoleGuard.jsx
│   │   ├── authorization/
│   │   │   ├── AuthorizationList.jsx
│   │   │   ├── AuthorizationCard.jsx
│   │   │   ├── AuthorizationForm.jsx
│   │   │   ├── AuthorizationDetails.jsx
│   │   │   ├── AlertBadge.jsx
│   │   │   └── ProgressBar.jsx
│   │   ├── usage/
│   │   │   ├── UsageForm.jsx
│   │   │   ├── UsageHistory.jsx
│   │   │   └── UsageStats.jsx
│   │   ├── product/
│   │   │   ├── ProductList.jsx
│   │   │   ├── ProductForm.jsx
│   │   │   └── ProductCard.jsx
│   │   ├── notification/
│   │   │   ├── NotificationBell.jsx
│   │   │   ├── NotificationList.jsx
│   │   │   └── NotificationItem.jsx
│   │   └── dashboard/
│   │       ├── StatCard.jsx
│   │       ├── AlertsWidget.jsx
│   │       ├── ChartWidget.jsx
│   │       └── RecentActivity.jsx
│   ├── pages/
│   │   ├── Login.jsx
│   │   ├── Dashboard.jsx
│   │   ├── Authorizations.jsx
│   │   ├── AuthorizationDetail.jsx
│   │   ├── UsageTracking.jsx
│   │   ├── Products.jsx
│   │   ├── Users.jsx
│   │   ├── Reports.jsx
│   │   └── NotFound.jsx
│   ├── context/
│   │   ├── AuthContext.jsx
```

```
│   │   └── NotificationContext.jsx
│   ├── hooks/
│   │   ├── useAuth.js
│   │   ├── useAuthorizations.js
│   │   ├── useProducts.js
│   │   ├── useNotifications.js
│   │   └── useDebounce.js
│   ├── utils/
│   │   ├── constants.js
│   │   ├── helpers.js
│   │   ├── calculations.js       # VBA logic in JS
│   │   ├── formatters.js
│   │   └── validators.js
│   ├── styles/
│   │   └── index.css             # Tailwind + custom styles
│   ├── App.jsx
│   └── index.js
├── .env.example
├── .gitignore
├── package.json
├── tailwind.config.js
└── postcss.config.js
```

(Continuing in next section...)

---

5. UI/UX AVEC TAILWIND CSS

5.1 Design System

tailwind.config.js

```
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        // Novo Nordisk Brand Colors (example)
        primary: {
          50: '#E6EBF5',
          100: '#CCD6EB',
          200: '#99AED7',
          300: '#6685C3',
          400: '#335DAF',
          500: '#001965', // Main brand color
          600: '#001451',
          700: '#000F3D',
          800: '#000A28',
          900: '#000514',
```

```
        },
        // Alert Colors (matching VBA logic)
        alert: {
          green: '#10B981',
          yellow: '#F59E0B',
          orange: '#F97316',
          red: '#EF4444',
        },
        // Status Colors
        status: {
          active: '#10B981',
          expired: '#EF4444',
          depleted: '#F97316',
          cancelled: '#6B7280',
        }
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
      },
    },
  },
  plugins: [
    require('@tailwindcss/forms'),
  ],
}
```

5.2 Components Réutilisables

components/common/Badge.jsx

```
import React from 'react';

const Badge = ({ children, variant = 'default', size = 'md', className = '' }) => {
  const variants = {
    // Alert levels (matching VBA)
    green: 'bg-alert-green text-white',
    yellow: 'bg-alert-yellow text-white',
    orange: 'bg-alert-orange text-white',
    red: 'bg-alert-red text-white',

    // Status
    active: 'bg-status-active text-white',
    expired: 'bg-status-expired text-white',
    depleted: 'bg-status-depleted text-white',
    cancelled: 'bg-status-cancelled text-white',

    // Default
    default: 'bg-gray-200 text-gray-800',
  };

  const sizes = {
    sm: 'px-2 py-0.5 text-xs',
```

```
    md: 'px-2.5 py-1 text-sm',
    lg: 'px-3 py-1.5 text-base',
  };

  return (
    <span
      className={`
        inline-flex items-center rounded-full font-medium
        ${variants[variant] || variants.default}
        ${sizes[size]}
        ${className}
      `}
    >
      {children}
    </span>
  );
};

export default Badge;
```

components/authorization/AlertBadge.jsx

```
import React from 'react';
import Badge from '../common/Badge';

/**
 * Alert Badge Component
 * Displays alert level with appropriate color (matching VBA logic)
 */
const AlertBadge = ({ alertLevel, showLabel = true }) => {
  const labels = {
    GREEN: 'Normal',
    YELLOW: 'Attention',
    ORANGE: 'Alerte',
    RED: 'Critique',
  };

  const variants = {
    GREEN: 'green',
    YELLOW: 'yellow',
    ORANGE: 'orange',
    RED: 'red',
  };

  return (
    <Badge variant={variants[alertLevel]} size="sm">
      <span className="flex items-center gap-1">
        <span className="w-2 h-2 rounded-full bg-white" />
        {showLabel && labels[alertLevel]}
      </span>
    </Badge>
```

```
  );
};

export default AlertBadge;
```

components/authorization/ProgressBar.jsx

```
import React from 'react';

/**
 * Progress Bar Component
 * Shows percentage used with color coding (matching VBA thresholds)
 */
const ProgressBar = ({ percentageUsed, percentageRemaining, daysRemaining }) => {
  const percentage = parseFloat(percentageUsed);

  // Determine color based on VBA thresholds
  let colorClass = 'bg-alert-green';
  if (percentageRemaining <= 20 || daysRemaining <= 7) {
    colorClass = 'bg-alert-red';
  } else if (percentageRemaining <= 50 || daysRemaining <= 30) {
    colorClass = 'bg-alert-orange';
  } else if (percentageRemaining <= 80 || daysRemaining <= 60) {
    colorClass = 'bg-alert-yellow';
  }

  return (
    <div className="w-full">
      <div className="flex justify-between text-sm mb-1">
        <span className="text-gray-600">Utilisé</span>
        <span className="font-semibold">{percentage.toFixed(1)}%</span>
      </div>
      <div className="w-full bg-gray-200 rounded-full h-2.5 overflow-hidden">
        <div
          className={`h-full ${colorClass} transition-all duration-300`}
          style={{ width: `${percentage}%` }}
        />
      </div>
      <div className="flex justify-between text-xs text-gray-500 mt-1">
        <span>0%</span>
        <span>100%</span>
      </div>
    </div>
  );
};

export default ProgressBar;
```

---

6. FONCTIONNALITÉS DÉTAILLÉES

6.1 Création d'Autorisation

Frontend Component

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { createAuthorization } from '../../api/authorization.api';
import { getProducts } from '../../api/product.api';
import { getUsers } from '../../api/user.api';
import { calculateExpirationDate } from '../../utils/calculations';

const AuthorizationForm = () => {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);
  const [products, setProducts] = useState([]);
  const [users, setUsers] = useState([]);

  const [formData, setFormData] = useState({
    productId: '',
    userId: '',
    authorizedQuantity: '',
    validityPeriodDays: '',
    purpose: '',
    notes: ''
  });

  const [calculatedExpiration, setCalculatedExpiration] = useState(null);

  useEffect(() => {
    loadData();
  }, []);

  useEffect(() => {
    // Calculate expiration date when validity period changes
    if (formData.validityPeriodDays) {
      const expiration = calculateExpirationDate(
        new Date(),
        parseInt(formData.validityPeriodDays)
      );
      setCalculatedExpiration(expiration);
    }
  }, [formData.validityPeriodDays]);

  const loadData = async () => {
    try {
      const [productsRes, usersRes] = await Promise.all([
        getProducts({ isActive: true }),
        getUsers({ role: 'UTILISATEUR' })
      ]);
```

```
      setProducts(productsRes.data);
      setUsers(usersRes.data);
    } catch (error) {
      console.error('Error loading data:', error);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);

    try {
      const response = await createAuthorization(formData);
      // Show success message
      alert('Autorisation créée avec succès');
      navigate('/authorizations');
    } catch (error) {
      console.error('Error creating authorization:', error);
      alert(error.response?.data?.message || 'Erreur lors de la création');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="max-w-2xl mx-auto bg-white rounded-lg shadow
p-6">
      <h2 className="text-2xl font-bold text-gray-900 mb-6">Nouvelle Autorisation</h2>

      {/* Product Selection */}
      <div className="mb-4">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Produit *
        </label>
        <select
          required
          value={formData.productId}
          onChange={(e) => setFormData({ ...formData, productId: e.target.value })}
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
focus:ring-primary-500"
        >
          <option value="">Sélectionnez un produit</option>
          {products.map(product => (
            <option key={product.id} value={product.id}>
              {product.name} ({product.casNumber}) - {product.unit}
            </option>
          ))}
        </select>
      </div>
```

```
      {/* User Selection */}
      <div className="mb-4">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Utilisateur *
        </label>
        <select
          required
          value={formData.userId}
          onChange={(e) => setFormData({ ...formData, userId: e.target.value })}
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
focus:ring-primary-500"
        >
          <option value="">Sélectionnez un utilisateur</option>
          {users.map(user => (
            <option key={user.id} value={user.id}>
              {user.firstName} {user.lastName} ({user.department})
            </option>
          ))}
        </select>
      </div>

      {/* Quantity */}
      <div className="mb-4">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Quantité autorisée *
        </label>
        <input
          type="number"
          step="0.001"
          required
          min="0.001"
          value={formData.authorizedQuantity}
          onChange={(e) => setFormData({ ...formData, authorizedQuantity: e.target.value
})}
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
focus:ring-primary-500"
        />
      </div>

      {/* Validity Period */}
      <div className="mb-4">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Période de validité (jours) *
        </label>
        <input
          type="number"
          required
          min="1"
          value={formData.validityPeriodDays}
          onChange={(e) => setFormData({ ...formData, validityPeriodDays: e.target.value
})}
```

```
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
 focus:ring-primary-500"
        />
        {calculatedExpiration && (
          <p className="mt-2 text-sm text-gray-600">
            Date d'expiration calculée: <strong>{new Date(calculatedExpiration).toLocaleD
ateString('fr-FR')}</strong>
          </p>
        )}
      </div>

      {/* Purpose */}
      <div className="mb-4">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Objectif *
        </label>
        <textarea
          required
          rows={3}
          value={formData.purpose}
          onChange={(e) => setFormData({ ...formData, purpose: e.target.value })}
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
 focus:ring-primary-500"
        />
      </div>

      {/* Notes */}
      <div className="mb-6">
        <label className="block text-sm font-medium text-gray-700 mb-2">
          Notes
        </label>
        <textarea
          rows={2}
          value={formData.notes}
          onChange={(e) => setFormData({ ...formData, notes: e.target.value })}
          className="w-full rounded-md border-gray-300 shadow-sm focus:border-primary-500
 focus:ring-primary-500"
        />
      </div>

      {/* Buttons */}
      <div className="flex gap-3 justify-end">
        <button
          type="button"
          onClick={() => navigate('/authorizations')}
          className="px-4 py-2 border border-gray-300 rounded-md text-gray-700 hover:bg-g
ray-50"
        >
          Annuler
        </button>
```

```
        <button
          type="submit"
          disabled={loading}
          className="px-4 py-2 bg-primary-500 text-white rounded-md hover:bg-primary-600
disabled:opacity-50"
        >
          {loading ? 'Création...' : 'Créer l\'autorisation'}
        </button>
      </div>
    </form>
  );
};

export default AuthorizationForm;
```

---

7. CALCULS ET LOGIQUE MÉTIER (VBA → JavaScript)

utils/calculations.js

```
/**
 * Calculations utility
 * Implements exact VBA logic in JavaScript
 */

/**
 * Calculate expiration date
 * VBA: CalculDateEcheance
 */
export const calculateExpirationDate = (startDate, validityPeriodDays) => {
  const date = new Date(startDate);
  date.setDate(date.getDate() + parseInt(validityPeriodDays));
  return date.toISOString().split('T')[0];
};

/**
 * Calculate days remaining
 * VBA: CalculJoursRestants
 */
export const calculateDaysRemaining = (expirationDate) => {
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const expiration = new Date(expirationDate);
  expiration.setHours(0, 0, 0, 0);

  const diffTime = expiration - today;
  const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
  return diffDays;
};
```

```
/**
 * Calculate percentage used
 * VBA: CalculPourcentageAcquis
 */
export const calculatePercentageUsed = (usedQuantity, authorizedQuantity) => {
  if (authorizedQuantity === 0) return 0;
  return ((usedQuantity / authorizedQuantity) * 100).toFixed(2);
};

/**
 * Calculate percentage remaining
 * VBA: CalculPourcentageRestant
 */
export const calculatePercentageRemaining = (usedQuantity, authorizedQuantity) => {
  const percentageUsed = calculatePercentageUsed(usedQuantity, authorizedQuantity);
  return (100 - parseFloat(percentageUsed)).toFixed(2);
};

/**
 * Determine alert level
 * VBA: DeterminerNiveauAlerte
 */
export const determineAlertLevel = (percentageRemaining, daysRemaining) => {
  const percentRemaining = parseFloat(percentageRemaining);
  const daysLeft = parseInt(daysRemaining);

  if (percentRemaining <= 20 || daysLeft <= 7) {
    return 'RED';
  } else if (percentRemaining <= 50 || daysLeft <= 30) {
    return 'ORANGE';
  } else if (percentRemaining <= 80 || daysLeft <= 60) {
    return 'YELLOW';
  } else {
    return 'GREEN';
  }
};

/**
 * Check if quantity would exceed authorization
 * VBA: ControleDepassement
 */
export const checkOverage = (requestedQuantity, remainingQuantity) => {
  const requested = parseFloat(requestedQuantity);
  const remaining = parseFloat(remainingQuantity);

  return {
    allowed: requested <= remaining,
    exceeded: Math.max(0, requested - remaining),
    remaining: remaining,
```

```
    requested: requested
  };
};

/**
 * Get alert color
 */
export const getAlertColor = (alertLevel) => {
  const colors = {
    'GREEN': '#10B981',
    'YELLOW': '#F59E0B',
    'ORANGE': '#F97316',
    'RED': '#EF4444'
  };
  return colors[alertLevel] || colors.GREEN;
};

/**
 * Get alert label
 */
export const getAlertLabel = (alertLevel) => {
  const labels = {
    'GREEN': 'Normal',
    'YELLOW': 'Attention',
    'ORANGE': 'Alerte',
    'RED': 'Critique'
  };
  return labels[alertLevel] || 'Inconnu';
};

/**
 * Format date to French format
 */
export const formatDateFR = (date) => {
  return new Date(date).toLocaleDateString('fr-FR', {
    day: '2-digit',
    month: '2-digit',
    year: 'numeric'
  });
};

/**
 * Format quantity with unit
 */
export const formatQuantity = (quantity, unit) => {
  const num = parseFloat(quantity);
  return `${num.toLocaleString('fr-FR', { maximumFractionDigits: 3 })} ${unit}`;
};
```

---

8. SYSTÈME DE NOTIFICATIONS

Backend Scheduler

services/scheduler.service.js

```
const cron = require('node-cron');
const alertService = require('./alert.service');
const { Authorization } = require('../models');
const winston = require('winston');

class SchedulerService {
  init() {
    // Check alerts every 6 hours
    cron.schedule(process.env.SCHEDULER_CHECK_ALERTS_CRON || '0 */6 * * *', async () => {
      winston.info('Running scheduled alert check...');
      try {
        await alertService.checkAllAuthorizationsForAlerts();
        winston.info('Alert check completed successfully');
      } catch (error) {
        winston.error('Alert check failed:', error);
      }
    });

    // Send daily email digest
    cron.schedule(process.env.SCHEDULER_SEND_EMAIL_CRON || '0 8 * * *', async () => {
      winston.info('Sending daily email digest...');
      try {
        await this.sendDailyDigest();
        winston.info('Daily digest sent successfully');
      } catch (error) {
        winston.error('Daily digest failed:', error);
      }
    });

    winston.info('Scheduler initialized');
  }

  async sendDailyDigest() {
    // Implementation for daily digest emails
    // Send summary of critical alerts to responsables
  }
}

module.exports = new SchedulerService();
```

---

9. SÉCURITÉ

Middleware d'Authentification

middleware/auth.js

```
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const authenticateToken = async (req, res, next) => {
  try {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Token d\'authentification requis'
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const user = await User.findByPk(decoded.userId);
    if (!user || !user.isActive) {
      return res.status(401).json({
        success: false,
        message: 'Utilisateur non trouvé ou inactif'
      });
    }

    req.user = {
      id: user.id,
      email: user.email,
      role: user.role,
      department: user.department
    };

    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        message: 'Token expiré',
        code: 'TOKEN_EXPIRED'
      });
    }

    return res.status(403).json({
      success: false,
      message: 'Token invalide'
```

```
    });
  }
};

module.exports = { authenticateToken };
```

---

10. DÉPLOIEMENT

Variables d'Environnement Production

```
NODE_ENV=production
PORT=5000

DB_HOST=your-db-host
DB_PORT=5432
DB_NAME=chemical_management_prod
DB_USER=prod_user
DB_PASSWORD=strong_password
DB_SSL=true

JWT_SECRET=your_production_jwt_secret_min_32_chars
JWT_EXPIRES_IN=24h

SMTP_HOST=smtp.company.com
SMTP_PORT=587
SMTP_USER=noreply@company.com
SMTP_PASSWORD=smtp_password

FRONTEND_URL=https://chemical-management.company.com

RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=100
```

Scripts de Déploiement

package.json (backend)

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "build": "echo 'No build step required'",
    "migrate": "sequelize-cli db:migrate",
    "migrate:prod": "NODE_ENV=production sequelize-cli db:migrate",
    "seed": "sequelize-cli db:seed:all",
    "test": "jest --coverage"
  }
}
```

FIN DU DOCUMENT TECHNIQUE

Ce document fournit une base complète pour développer l'application. Chaque section peut être étendue selon les besoins spécifiques.